

Are Language Models Actually Useful for Time Series Forecasting?

Mingtian Tan

University of Virginia
wtd3gz@virginia.edu

Mike A. Merrill

University of Washington
mikeam@cs.washington.edu

Vinayak Gupta

University of Washington
vinayak@cs.washington.edu

Tim Althoff

University of Washington
althoff@cs.washington

Thomas Hartvigsen

University of Virginia
hartvigsen@virginia.edu

Abstract

Large language models (LLMs) are being applied to time series forecasting. But are language models actually useful for time series? In a series of ablation studies on three recent and popular LLM-based time series forecasting methods, we find that removing the LLM component or replacing it with a basic attention layer does not degrade forecasting performance—in most cases, the results even improve! We also find that despite their significant computational cost, pretrained LLMs do no better than models trained from scratch, do not represent the sequential dependencies in time series, and do not assist in few-shot settings. Additionally, we explore time series encoders and find that patching and attention structures perform similarly to LLM-based forecasters.¹

1 Introduction

Time series analysis is a critical problem across many domains, including disease propagation forecasting [8], retail sales analysis [3], healthcare [25, 16] and finance [30]. A great deal of recent work in time series analysis (constituting repositories with more than 1200 total stars on GitHub) has focused on adapting pretrained large language models (LLMs) to classify, forecast, and detect anomalies in time series [14, 49, 21, 4, 5, 31, 13, 43, 15]. These papers posit that language models, being advanced models for sequential dependencies in text, may generalize to the sequential dependencies in time series data. This hypothesis is unsurprising given language models are now pervasive in machine learning research. However, direct connections between language modeling and time series forecasting remain largely undefined. So to what extent is language modeling *really* beneficial for traditional time series tasks?

Our claim is simple but profound: **popular LLM-based time series forecasters perform the same or worse than basic LLM-free ablations, yet require orders of magnitude more compute.** Derived from extensive ablations, this reveals a worrying trend in contemporary time series forecasting literature. Our goal is not to imply that language models will never be useful for time series. In fact, recent works point to many exciting and promising ways that language and time series interact, like time series reasoning [24, 7, 44, 41, 36], social understanding [6] and financial reasoning [35, 19]. Rather, we aim to highlight surprising findings that existing methods do very little to use the innate reasoning power of pretrained language models on established time series tasks.

We substantiate our claim by performing three ablations of three popular and recent LLM-based forecasting methods [49, 14, 21] using eight standard benchmark datasets from reference methods

¹All resources needed to reproduce our work are available: <https://github.com/BennyTMT/LLMsForTimeSeries>

and another five datasets from MONASH [12]. First, we successfully reproduce results from the original publications. Then, we show that replacing language models with simple attention layers, basic transformer blocks, randomly-initialized language models, and *even removing the language model entirely*, yields comparable or better performance. The same performance was observed on another five datasets that were not studied by the reference methods.

Next, we compare the training and inference speed of these methods against their ablations, showing that these simpler methods reduce training and inference time by up to three orders of magnitude while maintaining comparable performance. Then, to investigate the source of LLM forecaster’s performance, we further explore time series encoders. We find that a simple linear model with an encoder composed of patching and attention can achieve forecasting performance similar to that of LLMs. Next, we test whether the sequence modeling capabilities of LLMs transfer to time series by shuffling input time series and find no appreciable change in performance. Finally, we show that LLMs do not even help forecasting in few-shot settings with 10% of the training data. We discuss the implications of our findings and suggest that time series methods that use large language models are better left to multimodal applications [4, 11, 37] that require textual reasoning.

The key contributions we make in this paper are as follows:

- We propose three straightforward ablation methods for methods that pass time series into LLMs for forecasting. We then ablate three top-tier methods on thirteen standard datasets and find that LLMs fail to convincingly improve time series forecasting. However, they significantly increase computational costs in both training and inference.
- We study the impact of an LLM’s pretraining by re-initializing their weights prior to forecasting. We find that this has no impact on forecasting performance. Additionally, in shuffling input time series, we find no evidence the LLMs successfully transfer sequence modeling abilities from text to time series and no indication they help in few-shot settings.
- We find a very simple model, with patching and attention as encoder, can achieve performance similar to LLMs. This suggests a massive gap between the benefits LLMs pose and the time series forecasting problem, despite a rapid rush to adopt LLMs.

2 Related Work

Here, we summarize the key works relevant to LLM-based time series models. They can be broadly classified into three sections: (i) time series forecasting using LLMs; (ii) encoders in LLM time series models; and (iii) smaller and efficient neural models for time-series.

Time Series Forecasting Using LLMs. Recently, with the development of Large Language Models (LLMs) [10, 28, 33] and their demonstrated multi-modal capabilities, more researchers have successfully applied LLMs to time series forecasting tasks [13, 15, 5, 4]. Chang *et al.*, [5] used finetuning the transformer module and positional encoding in GPT-2 to align pre-trained LLMs with time series data for forecasting tasks. Zhou et al. [49] proposed a similar finetuning method, named “OneFitAll”, for time series forecasting with GPT-2. Additionally, Jin et al. [14] introduced a reprogramming method to align LLM’s Word Embedding with time series embeddings, showing good representation of time series data on LLaMA [33]. Similarly, CALF [21] and TEST [31] adapted word embeddings to enable LLMs to forecast time series data effectively. In addition to time-series forecasting models, Liu et al. [22] show that these models can be extended to classifying health-time series, such as heart-rate and daily-footsteps. These models have also been shown to outperform supervised neural models in few-shot settings.

Encoders in LLM Time Series Models. In order for an LLM to learn from text it must first be discretized and encoded as word tokens which are $1 \times d$ vectors [10, 28, 33]. Similarly, LLM-based methods for time series learn discrete time series tokens. One method is to segment the time series into overlapping patches, which effectively shortens the time series while retaining its features [14, 49, 5, 4, 27, 26]. Another method involves decomposing a method based on its trend, seasonal components, and residual components [4, 27]. Lastly, Liu et al. [21] feed the multivariate time series using a Transformer to enable different channels to learn the dynamics of other channels. These embedding procedures are followed by a linear neural network layer that projects the time series encoding to the same dimensions used by the pre-trained LLM.

Dataset	ETTh1 & ETTh2	ETTm1 & ETTm2	Traffic	Electricity	Weather	Illness
Channels	7	7	862	321	21	7
Sampling-Rate	1 Hour	15 Min.	1 Hour	1 Hour	10 Min.	1 Week
Timesteps	17,420	69,680	17,544	26,304	52,696	966

Table 1: Statistics for all datasets used in reference methods [49, 21, 14].

Method	Base Model	Learnable LM Parameters	Positional Embeddings	Align Word Embeddings	Multimodal
OneFitsAll [49]	GPT-2	Add&Norm	Fine-Tune	○	○
Time-LLM [14]	LLaMA	None	Freeze	●	●
CALF [21]	GPT-2	LoRA	Fine-Tune	●	○

Table 2: Three popular methods for time series forecasting with Large Language Models.

Small and Efficient Neural Forecasters. In addition to LLMs, there has been a large body of research on smaller yet efficient frameworks that outperform their bulky counterparts in time series forecasting [18, 46, 32, 23, 2]. For example, Zeng et al. [45] present DLinear, an incredibly simple model that combines decomposition techniques and achieves better forecasting performance than state-of-the-art transformer-based time series architectures at the time, such as Informer [47], FEDformer [48], and Autoformer [39]. Furthermore, Xu et al. [42] introduces a lightweight model with only 10k parameters, which captures both amplitude and phase information in the time-series to outperform transformer-based models.

3 Experimental Setup

We use three state-of-the-art methods for time series forecasting and propose three ablation methods for LLMs: (i) “w/o LLM”; (ii) “LLM2Attn”; (iii) and “LLM2Trsf”. To evaluate the effectiveness of LLMs in time series forecasting, we test these methods on eight standard datasets.

3.1 Reference Methods for Language Models and Time Series

We experiment with three recent methods for time series forecasting using LLMs. All models were published between December 2023 and May 2024 and are popular, with their GitHub repositories collectively amassing 1,245 stars. These methods are summarized in Table 2, and use either GPT-2 [28] or LLaMA [33] as base models, with different alignment and fine-tuning strategies.

- **OneFitsAll [49]:** OneFitsAll, sometimes called GPT4TS, applies instance norm and patching to the input time series and then feeds it into a linear layer to obtain a input representation for the language model. The multi-head attention and feed forward layers of the language model are frozen while the positional embeddings and layer norm are optimized during training. A final linear layer is used to transform the language model’s final hidden states into a prediction.
- **Time-LLM [14]:** In Time-LLM the input time series is tokenized via patching and aligned with a low-dimensional representation of word embeddings using multi-head attention. The outputs of this alignment, combined with the embeddings of descriptive statistical features, are passed to a frozen pre-trained language model. The output representations of the language model are then flattened and passed through a linear layer to obtain a forecast.
- **CALF [21]:** CALF embeds the input time series by treating each channel as a token. One half of the architecture is a “textual branch” which uses cross attention to align the time series representation with a low dimensional representation of the language model’s word embeddings. This representation is then passed through a pretrained, frozen language model to obtain a “textual prediction”. Simultaneously, a “temporal” branch learns a low-rank adapter for a pretrained language model based on the input time series to produce a “temporal prediction” which is used for inference. The model includes additional loss terms that enforce similarity between these representations.

Reproducibility Note. While experimenting with each model, we tried to replicate the conditions of their original papers. We used the original hyper-parameters, runtime environments, and code, including model architectures, training loops, and data-loaders. To ensure a fair comparison, we have included error metrics from the original papers alongside our results wherever possible.